

Vanilla system glossary

A catalog of the Cities: Skylines II systems, bindings and components a mod can tap into — what they're called, how to get hold of them, and what's actually been *confirmed* about each field as opposed to what the field name implies.

Check here before spending time rediscovering a system from scratch, and add to it whenever a mod in this repo pins down something new. See [HOWTO.md](#) for the architectural background (bindings, override seams, Harmony-vs-cooperative).

How to read this

Every claim below carries a confidence marker. **Nothing here should be trusted past its marker** — CS2 systems are internal APIs with no stability guarantee, and field layouts drift between game patches.

Marker	Meaning
[SDK]	Verified against the TypeScript definitions shipped in this repo (<code>MidnightToggle/types/*.d.ts</code>). Primary source — these come from the game's own UI SDK.
[GAME]	Confirmed by a mod in this repo actually running in-game.
[GUIDE]	From the cs2-modding-guide corpus, which derives it from published mods' source. Credible, but we haven't run it.
[?]	Named in a source but semantics, range, or exact signature unverified. Assume nothing; test before relying on it.

Part 1 — C# simulation systems

Unless stated otherwise, the instance is obtained the same way:

```
var system = World.GetOrCreateSystemManaged<TheSystem>();
```

`GetOrCreateSystemManaged<T>()` returns the existing instance or creates one, so it never returns null for a system the game defines — but it *silently succeeds* for a system that has been renamed or removed in a patch, handing you a fresh, inert instance. That's the single most common silent failure in CS2 modding.

Time, weather and the sky

PlanetarySystem

Owns the simulated clock and sun position. Time-of-day lighting derives from it.

Field	Type	Confidence	Behaviour
<code>overrideTime</code>	<code>bool</code>	[GAME]	The seam. <code>true</code> : the game stops computing time and uses <code>.time</code> instead. <code>false</code> : back to normal simulated time — no snapshot/restore needed, "off" just lets vanilla run.

time	float	[GAME] for 0f	0f is midnight (verified in-game by MidnightToggle). The full numeric domain is still unverified — whether it's a 0–24 hour scale, a 0–1 day fraction, or something else. Neither the guide nor our testing pins it down. Probe a second known time before assuming a range.
dayOfYear	float	[GUIDE]	Named as part of the same override group. Range/units unverified.
latitude	float	[GUIDE]	Sun-angle geography. Unverified.
longitude	float	[GUIDE]	Sun-angle geography. Unverified.

Used by: [MidnightToggle](#) — sets `overrideTime = true` and `time = 0f` while toggled on; clears `overrideTime` on toggle-off and in `OnDestroy`, and re-applies in `OnGameLoadingComplete` because the flag lives on a runtime system, not in the save file.

Cross-checked against: `rodrigmatrix/TimeWeatherAnarchy` (field names `overrideTime`, `time`).

ClimateSystem

Weather and season. Uses a **different override idiom** to `PlanetarySystem` — see [the two override idioms](#) below, this catches people out.

Each channel is a struct with a paired value and enable flag:

Channel	Confidence	Notes
---------	------------	-------

<code>temperature.overrideValue / .overrideState</code>	[GUIDE]	<code>float / bool</code>
<code>aurora.overrideValue / .overrideState</code>	[GUIDE]	<code>float / bool</code>
<code>cloudiness.overrideValue / .overrideState</code>	[GUIDE]	<code>float / bool</code>
<code>precipitation.overrideValue / .overrideState</code>	[GUIDE]	<code>float / bool</code>
<code>fog.overrideValue / .overrideState</code>	[GUIDE]	<code>float / bool</code>
<code>currentDate.overrideValue / .overrideState</code>	[GUIDE]	<code>float</code> , documented as a 0–1 year fraction
<code>currentSeasonName</code>	[GUIDE]	<code>string</code> , read- only, e.g. <code>"SeasonSummer"</code>

```
_climateSystem.temperature.overrideValue = targetValue;  
_climateSystem.temperature.overrideState = true;    // wit
```

Method: `SampleClimate(ClimatePrefab prefab, float t)` — overloaded, so a Harmony patch must disambiguate with `Type[] { typeof(ClimatePrefab), typeof(float) }`. **[GUIDE]**

Gotcha: retiming the day (see `TimeSystem`) does **not** retime climate; that needs its own patch. **[GUIDE]**

TimeSystem

The calendar/tick layer above `PlanetarySystem`. Mostly exposes getters rather than override seams, so mods that retime the day generally Harmony-patch it rather than writing fields. **[GUIDE]** throughout.

Member	Kind	Notes
kTicksPerDay	int const	Immutable baseline used for tick-rate maths.
normalizedDate, normalizedTime	float	Getters; common Harmony patch targets.
year	int	Getter.
GetYear(), GetDay(), GetTicks()	methods	
GetCurrentDateTime(), GetStartingDate()	methods	
GetElapsedYears(), GetTimeOfYear(), GetTimeOfDay()	methods	GetTimeOfDay() returns a fraction of a day.
m_Time, m_Date, m_Year	private fields	Reachable via Harmony Traverse. Private = no stability guarantee at all.

Related prefab component: `TimeSettingsData.m_DaysPerYear` (int) — cache the base value before scaling it. **[GUIDE]**

SimulationSystem

Game speed. **[GUIDE]**

Field	Type	Access	Notes
selectedSpeed	float	read/ write	The speed the user chose.
smoothSpeed	float	read- only	The eased, currently-applied speed. Changes every frame.

Pitfalls worth repeating:

- **Don't read speed from the UI-side binding**

(time.simulationSpeed\$) to drive logic — it can report pre-pause state while paused. Read selectedSpeed.

- smoothSpeed updates per frame; throttle any UI binding you publish from it (~500 ms) or you'll churn the React tree pointlessly.
- Writing selectedSpeed = 0 is allowed and produces a "paused but not paused" state. Clamp above 0 unless you want that.
- One mod clamps to [0, 8]; whether vanilla enforces its own bounds is unverified. [?]

Naming, prefabs and entities

NameSystem

Custom names on entities. [GUIDE]

Method	Notes
SetCustomName(Entity entity, string name)	Pass null to clear.
TryGetCustomName(Entity entity, out string customName)	
GetRenderedLabelName(Entity entity)	What's actually displayed (custom or vanilla).

The write does not repaint by itself. Add the CustomName and BatchesUpdated tag components after calling SetCustomName.

Scheduling hazard: vanilla name recomputation can clobber custom names on road edges in the same frame. Schedule after AggregateSystem in ModificationEnd:

```
updateSystem.UpdateAfter<YourSystem, AggregateSystem>(Sys
```

PrefabSystem

Reads and mutates prefab *templates* — so edits hit every instance of that prefab, and you cannot target one placed building this way.

[GUIDE]

Method	Notes
TryGetPrefab(PrefabID prefabID, out PrefabBase prefabBase)	Returns bool.
TryGetEntity(PrefabBase prefabBase, out Entity entity)	Returns bool.
TryGetComponentData<T>(PrefabBase prefabBase, out T component)	Returns bool.
AddComponentData<T>(PrefabBase prefabBase, T component)	Creates or overwrites.
UpdatePrefab(PrefabBase prefab)	Alternate write path for managed components (enums, flags).

Pitfalls: failed TryGet* calls fall through **silently** — log if it matters. Always scale from an immutable baseline, never the live field value, or repeated applies compound. Apply in OnGameLoadingComplete / settings-applied handlers, not OnCreate or a bare OnUpdate.

AggregateSystem

Vanilla producer for road aggregates. Not usually called directly — it matters as a **scheduling landmark**: anything writing edge names or aggregate metadata must run after it in ModificationEnd, or vanilla overwrites your work in the same frame. [GUIDE]

Tools, UI and rendering

ToolSystem / ToolBaseSystem / ToolRaycastSystem

[GUIDE] throughout.

- **ToolSystem** — global tool manager. `activeTool` (read/write) switches the active tool; assign `m_DefaultToolSystem` to cancel back to the default.
- **ToolBaseSystem** — subclass this to build a tool. Override `toolID`; call `base.OnCreate()` (it initialises `applyAction`); enable/disable `applyAction.shouldBeEnabled` in `OnStartRunning/OnStopRunning`; override `InitializeRaycast()` to configure filtering.
- **ToolRaycastSystem** — reachable as `m_ToolRaycastSystem` after `base.OnCreate()`. Fields: `typeMask` (e.g. `TypeMask.Net`, `TypeMask.StaticObjects`), `netLayerMask` (e.g. `Layer.Road | Layer.TrainTrack`), `raycastFlags` (e.g. `RaycastFlags.SubElements`, `RaycastFlags.Markers`).

Input actions expose `WasPressedThisFrame()`, `IsPressed()`, `WasReleasedThisFrame()` — the standard drag-select triple.

Highlighting gotcha: adding or removing the `Highlighted` component does nothing visually unless you also add `BatchesUpdated` on the same entity, every time, in both directions.

IconCommandSystem

Notification icons over buildings. `Game.Notifications`. [GUIDE]

Method	Notes
<code>CreateCommandBuffer()</code>	Returns a fresh <code>IconCommandBuffer</code> .
<code>AddCommandBufferWriter(JobHandle)</code>	Registers your job as a writer so the icon system waits on it.

`IconCommandBuffer.Remove(Entity building, Entity`

`iconPrefab`) — the second argument is the **notification prefab**, not the building; getting it wrong fails silently and leaves ghost icons. `Add(...)` exists but its signature needs per-version verification. [\[?\]](#)

Both lines are mandatory, in this order, or you get a silent race:

```
m_IconCommandSystem.AddCommandBufferWriter(cleanHandle);  
Dependency = cleanHandle;
```

OverlayRenderSystem

Shared gizmo buffer for lines, curves and circles. [\[GUIDE\]](#)

- `GetBuffer(out JobHandle ...)` — returns the overlay buffer. **Call `.Complete()` on the returned handle before writing**, or you race the game's own overlay jobs.
- `buffer.DrawCurve(color, curve, width, roundedLine)`, `buffer.DrawCircle(color, position, radius)`
- `GetTextMesh()`, `CopyFontAtlasParameters(...)` for TextMeshPro/SDF text.

For fully custom meshes,

`RenderPipelineManager.beginContextRendering` is a static event — `+=` in `OnCreate` and `-=` in `OnDestroy`, or the handler outlives the system and fires against disposed state. You own `Mesh/Material` lifetime when using `Graphics.DrawMesh`. Filter by `CameraType.Game / CameraType.SceneView` and bail early when your overlay is inactive.

UIManager — COUI host registration

How custom icon/image folders become `coui://` URLs. [\[GUIDE\]](#)

```
UIManager.defaultUISystem.AddHostLocation("mykey", Assemb
```

- `AddHostLocation(string key, string path)` — static

host

- `AddHostLocation(string uri, string path, bool shouldWatch)` — `shouldWatch` semantics (live reload?)
unverified [?]
- `RemoveHostLocation(string uri) /`
`RemoveHostLocation(string uri, string path)`

Host keys are **globally shared across all mods** — last registration wins, so pick a distinctive stable key and treat it as public API. Derive the path from your executing assembly rather than hard-coding it.

ChirperUISystem

`Game.UI.InGame`. Harmony-only — there's no cooperative seam here; you patch `GetMessageID(Entity chirp)` to substitute chirp text, and `LocalizationDictionary.TryGetValue` (with `[HarmonyPriority(Priority.First)]`, which is mandatory) to resolve custom keys. Reflected internals like `m_ChirpQuery / m_CreatedChirpQuery` need null guards — a field rename otherwise becomes a `NullReferenceException`. [GUIDE]

PlatformManager (+ AchievementTriggerSystem)

Achievements. Note this is a **singleton via `PlatformManager.instance`**, not `GetOrCreateSystemManaged`. [GUIDE]

Member	Notes
<code>achievementsEnabled</code>	<code>bool</code> . The game flips this to <code>false</code> at startup when mods are detected — and does so <i>frames after</i> your load-time write, so a single assignment is not enough; re-assert over a window.

EnumerateAchievements()	All platform achievements.
GetAchievement(AchievementId id, out IAchievement?)	
UnlockAchievement(AchievementId id) / ClearAchievement(AchievementId id)	
ResetAchievements()	Unrecoverable. Gate behind a confirmation UI.

Null-guard `PlatformManager.instance` on *every* access — the platform layer isn't guaranteed to exist when your code runs. Schedule re-assertion after `AchievementTriggerSystem`.

Economy, citizens and traffic

These are mostly *ordering landmarks and component surfaces* rather than override seams — you generally schedule around them or patch them. All **[GUIDE]**, and all flagged patch-sensitive by the source.

System	Namespace	
TaxSystem	Game.Simulation	Taxation pass. GetModifiedCo taxRates, dis applies district mo
PayWageSystem	Game.Simulation	Wage computation
ResourceExporterSystem	Game.Simulation	Export accounting
WorkProviderSystem	Game.Simulation	Rewrites WorkPr tick — a postfix on documented way

CitizenBehaviorSystem, CitizenTravelPurposeSystem, WorkerSystem, LeisureSystem, StudentSystem, DeathCheckSystem	Game.Simulation	Citizen behaviour <i>replaced</i> wholesa
ResidentAISystem (+ .Actions), TripNeededSystem, ResourceBuyerSystem	Game.Simulation	Movement/trip/sho Finding disables.
TourismSystem, TouristSpawnSystem, AttractionSystem	Game.Simulation	Tourism.
PathUtils	Game.Pathfind	Static pathfinding lists are called o thing to break be verify every type

Useful static utilities: `EconomyUtils.GetMarketPrice / GetIndustrialPrice / GetServicePrice` (all static, Harmony-postfix-able) and `SimulationUtils.GetUpdateFrame (frameIndex, updatesPerDay, seed)` for spreading work across update slots.

Disabling a vanilla system

```
World.DefaultGameObjectInjectionWorld.GetOrCreateSystemMa
```

The instance stays in the world and stays discoverable; it just stops running. Then register your replacement in the *same* `SystemUpdatePhase`. Caveats: it's all-or-nothing (every effect of that system stops), two mods disabling the same system will fight, your decompiled clone freezes at the version you copied, and disabling a renamed system silently does nothing. **[GUIDE]**

The two override idioms

The two most-used systems use **different** override shapes, which is an easy source of "why is nothing happening":

System	Idiom	Example
PlanetarySystem	one bool flag + the value field	<code>overrideTime = true;</code> <code>time = 0f;</code>
ClimateSystem	per-channel overrideValue + overrideState	<code>temperature.overrideValue = 20f;</code> <code>temperature.overrideState = true;</code>

Setting `overrideValue` alone does nothing — `overrideState` must also be `true`. [\[GUIDE\]](#)

Both are *cooperative*: setting the flag back to `false` restores vanilla behaviour with nothing to snapshot or restore. Always clear your overrides in `OnDestroy` so unloading the mod never leaves the game stuck, and re-apply them in `OnGameLoadingComplete` since these flags live on runtime systems and are not saved.

Part 2 — The TSX side: `cs2/` bindings

This is the most under-appreciated surface in CS2 modding. The game ships a large set of vanilla UI bindings that a mod can read and, in many cases, write **without writing a single line of C#**. If what you want is already here, you don't need a `UISystemBase` at all.

All [\[SDK\]](#) — taken from `MidnightToggle/types/bindings.d.ts`

(3,646 lines, worth browsing directly).

```
import { time, cityInfo } from "cs2/bindings";
import { useValue } from "cs2/api";

const paused = useValue(time.simulationPaused$);
const money = useValue(cityInfo.money$);
time.setSimulationSpeed(2);
```

Namespaces

budget, camera, chirper, cinematic, cityInfo, climate, devTree, economyBudget, event, feature, game, infoview, infoviewTypes, life, loan, map, milestone, photo, policy, prefab, prefabEffects, prefabProperties, prefabRequirements, production, radio, selectedInfo, service, signatureBuilding, statistics, taxation, time, tool, toolbar, toolbarBottom, transport, tutorial, upgrade.

The ones worth knowing

Namespace	Notable members
time	ticks\$, day\$, timeSettings\$, lightingState\$, simulationPaused\$, simulationSpeed\$, simulationPausedBarrier\$; setSimulationPaused(), setSimulationSpeed(); helpers calculateDateFromTicks, calculateDateTimeFromTicks, calculateMinutesSinceMidnightFromTicks, calculateTimeFromMinutesSinceMidnight

climate	seasonName\$, weather\$, temperature\$ (read-only view of what ClimateSystem computes)
cityInfo	cityName\$, money\$, moneyDelta\$, population\$, populationDelta\$, unlimitedMoney\$; setCityName()
tool	activeTool\$, selectTool(), selectToolMode(), brush state (brushSize\$, brushStrength\$, brushAngle\$...), elevation\$, undergroundMode\$, snapOptionNames\$, isEditor\$, and tool id constants DEFAULT_TOOL, BULLDOZE_TOOL, NET_TOOL, OBJECT_TOOL, ZONE_TOOL, TERRAIN_TOOL, WATER_TOOL, ROUTE_TOOL, AREA_TOOL, UPGRADE_TOOL, SELECTION_TOOL
game	activeGameScreen\$, activeGamePanel\$, blockingPanelActive\$, canUseSaveSystem\$, plus showGamePanel()/toggleGamePanel()/closeGamePanel() and the full panel-type enums
selectedInfo	selectedEntity\$, activeSelection\$, titleSection\$, topSections\$/middleSections\$/bottomSections\$, selectEntity(), clearSelection() — the selected-info panel surface, and the biggest single namespace
cityInfo / infoview	infoview metrics: population\$, unemployment\$, homeless\$, birthRate\$/deathRate\$, electricityProduction\$/Consumption\$, waterCapacity\$, crimeProbability\$, trafficFlow\$, averageHealth\$, garbageProductionRate\$... (hundreds of read-only city statistics)
taxation	areaTaxRates\$, setTaxRate(), setAreaTaxRate(), setResourceTaxRate(), minTaxRate/maxTaxRate

policy	cityPolicies\$, setCityPolicy(), setPolicy()
transport	transportLines\$, renameLine(), setLineColor(), setLineActive(), deleteLine()
chirper	chirps\$, chirpAdded\$, addLike(), removeLike()
production	resource/production chain maps: resources\$, services\$, productionChainData\$, storedResource\$
camera / photo / cinematic	free camera, photo mode widgets, cinematic keyframe sequences

Binding kinds: `ValueBinding<T>` (subscribe with `useValue`), `MapBinding<K, V>` (keyed, e.g. `resourceDetails$` by `Entity`), and `EventBinding<T>` (fires on occurrence, e.g. `chirpAdded$`). **[SDK]**

Caveat carried over from `SimulationSystem`: these are *UI* bindings. For control logic prefer the C#-side field where one exists — the guide specifically warns that `time.simulationSpeed$` can read stale while paused. **[GUIDE]**

cs2/api — the full surface

More than the four functions the HOWTO covers. All **[SDK]**, from `types/api.d.ts`:

Function	Purpose
<code>bindValue<T>(group, name, fallbackValue?)</code>	Handle to a C# <code>ValueBinding</code> .
<code>bindLocalValue<T>(initialValue)</code>	Purely client-side binding, no C# counterpart.

<code>bindTrigger(group, name)</code>	Returns a ready-made <code>() => void</code> caller.
<code>bindTriggerWithArgs<T[]>(group, name)</code>	Same, with arguments.
<code>trigger(group, name, ...args)</code>	Fire-and-forget into C#.
<code>call<T>(group, name, ...args)</code>	Request/response; returns <code>Promise<T></code> .
<code>useValue<V>(binding)</code>	Subscribe + re-render.
<code>useValueRef<V>(binding)</code>	Subscribe into a ref without re-rendering — use for high-frequency values.
<code>useValueOnChange<V>(binding, onChange, depth?)</code>	Subscribe with a change callback.

Part 3 — UI mount points (`moduleRegistry`)

[SDK], from `types/modding.d.ts`. Note this list is **authoritative and differs from the guide**, which omits `UniversalModMenu` and `hasAppend`:

```
type AppendHookTargets =
  "Menu" | "Editor" | "Game" | "GameTopLeft" | "GameTopRight"
  | "GameBottomRight" | "UniversalModMenu";
```

There is **no `GameBottomLeft`** — that slot does not exist.

Method	Signature
--------	-----------

append	append(target: AppendHookTargets, appendedComponent, index?) — or append(modulePath, exportName, appendedComponent?, index?) for a specific vanilla module
hasAppend	hasAppend(target: AppendHookTargets): boolean
extend	extend(modulePath, exportNameOrSCSSValue, extendCb?) — wrap/replace a vanilla component
override	override(modulePath, exportName, newValue)
get / add	get(modulePath, exportName) / add(modulePath, module)
find	find(query: string RegExp): [path, ...exports][] — how you discover vanilla module paths at runtime
registry	Map<string, Record<string, any>> — the raw module map

index on `append` controls ordering among the mods sharing that slot.

Known vanilla module paths used with `extend()` **[GUIDE]**: `game-ui/game/components/tool-options/mouse-tool-options/mouse-tool-options.tsx` (export `MouseToolOptions`), and `asset-category-tab-bar.tsx` (export `AssetCategoryTabBar`). Use `moduleRegistry.find(/pattern/)` to discover others rather than guessing.

Part 4 — SystemUpdatePhase reference

Where to register a system in `Mod.OnLoad`. **[GUIDE]**

Phase	Use for
<code>GameSimulation</code>	Main gameplay simulation; systems advancing city state each tick.
<code>EditorSimulation</code>	Simulation while in map/asset editor.
<code>UIUpdate</code>	Publishing state to Gameface bindings. The right phase for a <code>UISystemBase</code>.
<code>UITooltip</code>	Tooltip generation.
<code>ToolUpdate</code>	Per-frame tool input/preview.
<code>Modification1</code>	Earliest world/network modification step.
<code>Modification2B</code>	Aligned with vanilla <code>SubObjectSystem</code> .
<code>Modification4B</code>	Intermediate; aligned with <code>TrafficLightInitializationSystem</code> .
<code>Modification5</code>	Late modification step.
<code>ModificationEnd</code>	Tail of the modification pipeline, after geometry/aggregate edits. Where <code>AggregateSystem</code> and <code>LanesModifiedSystem</code> land — schedule name/metadata writes after them here.
<code>Rendering</code>	Overlays, meshes, fonts.
<code>PreCulling</code>	Visibility data before culling.
<code>PrefabUpdate</code>	Reapplying prefab-derived data.
<code>PrefabReferences</code>	Prefab reference resolution during init.
<code>Cleanup</code>	End-of-frame disposal.
<code>Serialize / Deserialize</code>	Save / load.

Discrepancy to resolve [?]: the guide's phase list above doesn't include `MainLoop`, yet its own achievement recipe schedules with `SystemUpdatePhase.MainLoop`. One of the two is out of date. Check

IntelliSense against the real `SystemUpdatePhase` enum before using either name.

Ordering helpers: `updateSystem.UpdateAt<T>(phase)`, `UpdateBefore<TSelf, TOther>(phase)`, `UpdateAfter<TSelf, TOther>(phase)`.

Part 5 — ECS component quick reference

Components a mod reads or writes, grouped by namespace. **[GUIDE]** throughout, and field layouts are the *first* thing to drift between patches — treat every row as needing re-verification against the current game version.

Game.Citizens

`Citizen` (carries `CitizenAge`: `Child/Teen/Adult/Elderly`), `HouseholdMember`, `Household`, `HouseholdCitizen` (buffer, writable), `HouseholdNeed`, `HomelessHousehold/`
`TouristHousehold` (tags), `CurrentBuilding`, `CurrentTransport`, `Worker`, `Student`, `CarKeeper`, `BicycleOwner`, **`HealthProblem`** (writable — the key illness/injury/death hook), **`TripNeeded`** (buffer, writable — pending trips), `TravelPurpose`, `Leisure`, `AttendingMeeting`, `CoordinatedMeeting`, `MailSender`, `Criminal`.

Game.Companies

`WorkProvider` (writable; `m_MaxWorkers` — staffing cap), `Employee` (buffer), `ResourceBuyer` (writable), `ServiceAvailable`.

Game.Economy

Resources (buffer, writable — per-entity resource ledger), Resource (enum: Money, Garbage, LocalMail, OutgoingMail, UnsortedMail, ...), EconomyParameterData (singleton; **field layout explicitly unverified per patch [?]**).

Game.Net

CarLane (m_SpeedLimit, m_DefaultSpeedLimit, m_Flags), TrackLane (m_SpeedLimit), NetCarLane (m_Flags), CarLaneFlags.PublicOnly, Edge, Road, Curve, SubLane (buffer), Aggregated, AggregateElement (buffer), TrafficLights, LaneSignal.

Speed units gotcha [GUIDE]: lane m_SpeedLimit fields are in **2× m/s** (convert from km/h by /1.8), while the prefab-level RoadData.m_SpeedLimit is treated as plain m/s. Mixing these up silently doubles or halves speeds.

Game.Pathfind

PathInformation (m_State as PathFlags, m_Destination), PathElement (buffer), PathSpecification (m_Costs), PathfindCosts (m_Value, float4), PathFlags.Pending. Full layouts unverified beyond the cited fields [?].

Game.Prefabs

PrefabRef, RoadData (m_SpeedLimit), TimeSettingsData (m_DaysPerYear), PathfindCarData (m_TurningCost, m_UnsafeTurningCost, m_CurveAngleCost, m_UTurnCost, m_UnsafeUTurnCost), TransportStopData (m_BoardingTime, m_TransportType), PublicTransportVehicleData (m_PassengerCapacity), TrafficSpawnerData (m_SpawnRate), PostFacilityData (m_PostVanCapacity, m_PostTruckCapacity, m_SortingRate), PostVanData

(m_MailCapacity), TransportType (enum: Train, Ship, Airplane, Bus, Tram, Subway, Ferry).

Game.Areas

CurrentDistrict (m_District), DistrictModifier (buffer), DistrictModifierType (enum: CarReserveProbability, BikeProbability, ...), Node (buffer, boundary geometry), SubArea (buffer, m_Area). AreaUtils.ApplyModifier(ref num, bufferData, DistrictModifierType) folds modifiers into a running value.

Policies are a gap [?]: the guide states no dossier confirms the *policy-application* system — PolicySystem, Game.Policies.*, ActivePolicy and how policies map onto district modifier buffers are all unverified from C#. The TSX-side policy namespace (cityPolicies\$, setCityPolicy()) is [SDK]-confirmed and is currently the better-documented route.

Game.Buildings / Game.Vehicles / Game.Common

Building, PropertyRenter, Abandoned, GarbageProducer (writable), PostFacility; CarCurrentLane, GarbageTruck (m_State); Deleted (cascade-deletion marker), Highlighted + BatchesUpdated (see tools above), CustomName.

Game.Routes

CurrentRoute, WaitingPassengers (m_AverageWaitingTime, m_Count).

Sources

- **Primary (strongest):** the SDK typings in [MidnightToggle/types/](#) — `bindings.d.ts`, `api.d.ts`, `modding.d.ts`, `ui.d.ts`, `input.d.ts`, `ll0n.d.ts`. These ship with the UI mod template and describe the real runtime surface. Prefer them over any prose documentation, including this file.
- [BrokeAssSoftware/cs2-modding-guide](#) (integration branch) — 59 technique families, per-system reference pages and 23 case studies mined from published mods. Everything marked **[GUIDE]** here came from `how-to/recipes/*`, `reference/game-systems/*` and `reference/glossary.md`.
- `rodrigmatrix/TimeWeatherAnarchy` — `PlanetarySystem` field names.

Adding an entry

Keep the shape: how to obtain the instance, a field/method table with a confidence marker per row, which mod in this repo exercises it, and what's explicitly *not* verified. The value of this file is that it distinguishes "confirmed" from "plausible" — an entry that blurs the two is worse than no entry.